

OtoDecks Application Code:

Main.cpp:

```
/* NOT DONE BY ME AS THIS WAS INCLUDED IN THE PROJECT FOLDER WHEN  
WE STARTED BUILDING IT
```

```
/*
```

```
=====
```

This file contains the basic startup code for a JUCE application.

```
=====
```

```
*/
```

```
#include <JuceHeader.h>  
#include "MainComponent.h"
```

```
//=====
```

```
class OtoDecksApplication : public juce::JUCEApplication  
{  
public:
```

```
//=====
```

```
OtoDecksApplication() {}
```

```
const juce::String getApplicationName() override { return  
ProjectInfo::projectName; }
```

```
const juce::String getApplicationVersion() override { return  
ProjectInfo::versionString; }
```

```
bool moreThanOneInstanceAllowed() override { return true; }
```

```
//=====
=====
void initialise (const juce::String& commandLine) override
{
    // This method is where you should put your application's initialisation
    code..

    mainWindow.reset (new MainWindow (getApplicationName()));
}

void shutdown() override
{
    // Add your application's shutdown code here..

    mainWindow = nullptr; // (deletes our window)
}

//=====
=====
void systemRequestedQuit() override
{
    // This is called when the app is being asked to quit: you can ignore this
    // request and let the app carry on running, or call quit() to allow the app to
    close.
    quit();
}

void anotherInstanceStarted (const juce::String& commandLine) override
{
    // When another instance of the app is launched while this one is running,
    // this method is invoked, and the commandLine parameter tells you what
    // the other instance's command-line arguments were.
}
```

```
//=====
=====
/*
    This class implements the desktop window that contains an instance of
    our MainComponent class.
*/
class MainWindow : public juce::DocumentWindow
{
public:
    MainWindow (juce::String name)
        : DocumentWindow (name,
                          juce::Desktop::getInstance().getDefaultLookAndFeel()
                          .findColour
(juce::ResizableWindow::backgroundColourId),
                          DocumentWindow::allButtons)
    {
        setUsingNativeTitleBar (true);
        setContentOwned (new MainComponent(), true);

        #if JUCE_IOS || JUCE_ANDROID
            setFullScreen (true);
        #else
            setResizable (true, true);
            centreWithSize (getWidth(), getHeight());
        #endif

        setVisible (true);
    }

    void closeButtonPressed() override
    {
        // This is called when the user tries to close this window. Here, we'll just
        // ask the app to quit when this happens, but you can change this to do
        // whatever you need.
        JUCEApplication::getInstance()->systemRequestedQuit();
    }
}
```

```
/* Note: Be careful if you override any DocumentWindow methods - the
base
    class uses a lot of them, so by overriding you might break its
functionality.
    It's best to do all your work in your content component instead, but if
    you really have to override any DocumentWindow methods, make sure
your
    subclass also calls the superclass's method.
*/
```

```
private:
    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR
(MainWindow)
};
```

```
private:
    std::unique_ptr<MainWindow> mainWindow;
};
```

```
//=====
=====
// This macro generates the main() routine that launches the app.
START_JUCE_APPLICATION (OtoDecksApplication)

*/
```

MainComponent.h:

```
/* DONE BY ME
// Prevents the file from being included multiple times
#pragma once

// The JUCE framework
#include <JuceHeader.h>
```

```

// DJ Audio Player Header
#include "DJAudioPlayer.h"

// The GUI for each Deck
#include "DeckGUI.h"

// The Playlist Component header file
#include "PlaylistComponent.h"

//=====

// Handles callbackks and GUI
class MainComponent : public juce::AudioAppComponent
{
public:

    // The Constructor
    MainComponent();

    // The Destructor
    ~MainComponent() override;

    // Called before the beginning of the audio
    void prepareToPlay(int samplesPerBlockExpected, double sampleRate)
    override;

    // Called to fill audio buffers
    void getNextAudioBlock(const juce::AudioSourceChannelInfo&
    bufferToFill) override;

    // Called when the audio stops
    void releaseResources() override;

    // Draw the Graphics on the screen
    void paint(juce::Graphics& design) override;

```

```

// Reposition's UI Components
void resized() override;

// Save Equaliser settings
void storeEqualiser(const juce::String& trackPath, double lowGain, double
midGain, double highGain);

private:

// Manages audio formats
juce::AudioFormatManager formatManager;

// Cache used for thumbnails
juce::AudioThumbnailCache thumbCache{ 100 };

*/

/* NOT DONE BY ME
// Returns the JSON files used to store cue points
juce::File getCuesFile() const;

// Loads the cue data from the JSON file
juce::var loadCuesJson() const;

// Saves the cue data
void saveCuesJson(const juce::var& v) const;

// Applies Cues to Player
void applyCuesToPlayer(const juce::String& trackPath, DJAudioPlayer*
player);

// Stores the cues for a track
void storeCuesForTrack(const juce::String& trackPath, const
std::array<double, 8>& cues);

// Applies saved Equaliser values

```

```

        void applyEqualiserToPlayer(const juce::String& trackPath,
        DJAudioPlayer* player);

    /*
    /* DONE BY ME
        // The first audio player
        DJAudioPlayer player1{ formatManager };

        // The first deck
        DeckGUI deck1{ &player1, formatManager, thumbCache };

        // The second audio player
        DJAudioPlayer player2{ formatManager };

        // The seocnd deck
        DeckGUI deck2{ &player2, formatManager, thumbCache };

        // Playlist component
        PlaylistComponent playlistComponent{ formatManager };

        // Mixes audio players
        juce::MixerAudioSource mixerSource;

        // Prevents copying and leak detection

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(MainComponent)
    };

    */

```

MainComponent.cpp:

```

/* DONE BY ME
// The MainComponent Header file
#include "MainComponent.h"

```

```
//=====
=====
```

```
// The constructor
```

```
MainComponent::MainComponent() : playlistComponent(formatManager)
```

```
{
```

```
    setSize(800, 600);
```

```
    // Adds UI components
```

```
    addAndMakeVisible(deck1);
```

```
    addAndMakeVisible(deck2);
```

```
    addAndMakeVisible(playlistComponent);
```

```
    // 0 audio inputs, 2 outputs
```

```
    setAudioChannels(0, 2);
```

```
    // Registers basic audio formats
```

```
    formatManager.registerBasicFormats();
```

```
    // loads the playlist library
```

```
    playlistComponent.loadLibrary();
```

```
    // A user component calls this callback when a track is loaded into the deck
```

```
    playlistComponent.onLoadToDeck = [this](juce::File file, int deckIndex)
```

```
    {
```

```
        // if Deck 1
```

```
        if (deckIndex == 1)
```

```
        {
```

```
            // Load file into deck1
```

```
            deck1.loadFile(juce::URL{ file });
```

```
    */
```

```
/* NOT DONE BY ME
```

```
    // Restore previously saved Equaliser settings
```

```
    applyEqualiserToPlayer(file.getFullPathName(), &player1);
```

```
    }
```

```
    // if Deck 2
```



```

else if (deckIndex == 2)
{
    // Load file into deck 2
    deck2.loadFile(juce::URL{ file });

    // Restore the previously saved Equaliser Settings
    applyEqualiserToPlayer(file.getFullPathName(), &player2);
}
};

deck1.setDeckIndex(1);
deck2.setDeckIndex(2);

// Set the cue callback for deck 1
deck1.setCueCallbacks(

    // When deck 1 changes or saves hot cues, store them by their path
    [this](const juce::String& path, int, const std::array<double, 8>& cues)
    {
        storeCuesForTrack(path, cues);
    },
    // when deck 1 want to restore the same cues. load them and apply to the
player
    [this](const juce::String& path, int, DJAudioPlayer* player)
    {
        applyCuesToPlayer(path, player);
    }
);

// Same cue callbacks for deck 2
deck2.setCueCallbacks(
    [this](const juce::String& path, int, const std::array<double, 8>& cues)
    {
        storeCuesForTrack(path, cues);
    },
    [this](const juce::String& path, int, DJAudioPlayer* player)
    {

```

```

        applyCuesToPlayer(path, player);
    }
};

// When deck1 or Equaliser sliders change, store the values
deck1.onEQChanged = [this](const juce::String& path, double low, double mid,
double high)
{
    storeEqualiser(path, low, mid, high);
};

// Same for deck 2
deck2.onEQChanged = [this](const juce::String& path, double low, double mid,
double high)
{
    storeEqualiser(path, low, mid, high);
};
}
*/

/* DONE BY ME
// The Destructor
MainComponent::~MainComponent()
{
    // Save playlist to disk before closing'
    playlistComponent.saveLibrary();

    // Stops audio device and disconnects
    shutdownAudio();
}

// Called by JUCE whne audio device starts
void MainComponent::prepareToPlay(int samplesPerBlockExpected, double
sampleRate)
{
    // Add both players as inputs to the mixer
    mixerSource.addInputSource(&player1, false);

```

```

mixerSource.addInputSource(&player2, false);

// Prepare the mixer
mixerSource.prepareToPlay(samplesPerBlockExpected, sampleRate);

// This prepares both players too
player1.prepareToPlay(samplesPerBlockExpected, sampleRate);
player2.prepareToPlay(samplesPerBlockExpected, sampleRate);
}

// Get the next audio block
void MainComponent::getNextAudioBlock(const juce::AudioSourceChannelInfo&
bufferToFill)
{
    // Mixer fills the output buffer
    mixerSource.getNextAudioBlock(bufferToFill);
}

// Called when audio stops
void MainComponent::releaseResources()
{
    // Remove all inputs from the mixer
    mixerSource.removeAllInputs();

    // Release any resources
    mixerSource.releaseResources();

    // Release resources used by each player
    player1.releaseResources();
    player2.releaseResources();
}

//=====

=====

// Paint that is used for custom drawing

```

```

void MainComponent::paint(juce::Graphics& design)
{
    // Fill background with the default colour

    design.fillAll(getLookAndFeel().findColour(juce::ResizableWindow::backgroundColourId));
}

void MainComponent::resized()
{
    // Top half being decks
    deck1.setBounds(0, 0, getWidth() / 2, getHeight() / 2);
    deck2.setBounds(getWidth() / 2, 0, getWidth() / 2, getHeight() / 2);

    // Bottom half
    playlistComponent.setBounds(0, getHeight() / 2, getWidth(), getHeight() / 2);

    // Debug Message
    DBG("MainComponent::resized");
}
*/

/* NOT DONE BY ME
// Returns the file path where you store cues and equalisers
juce::File MainComponent::getCuesFile() const
{
    // Get user app-data directory
    auto dir =
juce::File::getSpecialLocation(juce::File::userApplicationDataDirectory).getChildFile(ProjectInfo::projectName);

    // Create directory if missing
    if (!dir.exists())
    {
        dir.createDirectory();
    }
}

```

```

    // Return the final JSON file path
    return dir.getChildFile("cues.json");
}

// Loads cues JSON into a juce::var structure
juce::var MainComponent::loadCuesJson() const
{
    // Get JSON file location
    auto f = getCuesFile();

    // if the file doesn't exist, return an empty object
    if (!f.existsAsFile())
    {
        return juce::var(new juce::DynamicObject());
    }

    // Parse JSON text
    auto parsed = juce::JSON::parse(f.loadFileAsString());

    // Check if the parsing failed, then return an object
    if (parsed.isVoid() || parsed.isUndefined())
    {
        return juce::var(new juce::DynamicObject());
    }

    // If the object is not parsed, return a new juce::DynamicObject
    if (!parsed.isObject())
    {
        return juce::var(new juce::DynamicObject());
    }

    // Return the parsed object
    return parsed;
}

// Saves a juce::var as JSON
void MainComponent::saveCuesJson(const juce::var& v) const

```

```

{
    // Convert var -> JSON
    getCuesFile().replaceWithText(juce::JSON::toString(v));
}

// Store 8 hot cues
void MainComponent::storeCuesForTrack(const juce::String& trackPath, const
std::array<double, 8>& cues)
{
    // Load current JSON root
    auto root = loadCuesJson();

    // Get DynamicObject pointer
    auto* object = root.getDynamicObject();

    // If root isn't an object, do nothing
    if (!object)
    {
        return;
    }

    // JUCE JSON arrays
    juce::Array<juce::var> cueArray;

    // Copy the array to the JUCE var array
    for (auto c : cues)
    {
        cueArray.add(c);
    }

    // Store key = trackPath
    object->setProperty(trackPath, juce::var(cueArray));

    // Save JSON back
    saveCuesJson(root);
}

```

```

// =====

// Restore cues for a track and apply them
void MainComponent::applyCuesToPlayer(const juce::String& trackPath,
DJAudioPlayer* player)
{
    // If player point is null, abort
    if (!player)
    {
        return;
    }

    // Loads JSON root
    auto root = loadCuesJson();
    auto* object = root.getDynamicObject();

    // if it's not a valid object, abort
    if (!object)
    {
        return;
    }

    // Get the property stored
    auto v = object->getProperty(trackPath);

    // if the propert isn't an array, then theres no stored cues
    if (!v.isArray())
    {
        return;
    }

    // Get the pointer to the JUCE var array
    auto* array = v.getArray();

    if (!array)
    {

```

```

        return;
    }

    // Create cues array and default all values
    std::array<double, 8> cues;
    cues.fill(-1.0);

    // Copy up to 8 values from JSON array
    for (int i = 0; i < juce::jmin(8, array->size()); ++i)
    {
        cues[(size_t)i] = (double)(*array)[i];
    }

    // Apply cues to the player
    player->setHotCues(cues);
}

// Store Equaliser Values
void MainComponent::storeEqualiser(const juce::String& trackPath, double low,
double mid, double high)
{
    // Load current JSON root
    auto root = loadCuesJson();

    // Get Dynamic Object
    auto* object = root.getDynamicObject();

    // Create a 3 value array for Equaliser
    juce::Array<juce::var> equaliserValues;
    equaliserValues.add(low);
    equaliserValues.add(mid);
    equaliserValues.add(high);

    // Store Equalisers a different key so it doesn't overwrite the cue array
    object->setProperty(trackPath + "_eq", juce::var(equaliserValues));

    // Save JSON back to disk

```



```

    saveCuesJson(root);
}

// Load Equaliser values for a track
void MainComponent::applyEqualiserToPlayer(const juce::String& trackPath,
DJAudioPlayer* player)
{
    // Load JSON
    auto root = loadCuesJson();

    // Get Dynamic Object
    auto* object = root.getDynamicObject();

    // Check if the Equaliser key exists
    if (object && object->hasProperty(trackPath + "_eq"))
    {
        auto equaliserVariable = object->getProperty(trackPath + "_eq");

        // Ensure it is an array
        if (equaliserVariable.isArray())
        {
            // Get array pointer
            auto* array = equaliserVariable.getArray();

            // Apply to player
            player->setEQ((double)(*array)[0], (double)(*array)[1],
(double)(*array)[2]);
        }
    }
}

*/

```

DJAudioPlayer.h:

```

/* DONE BY ME
// prevents header from being included more than once

```

```
#pragma once
```

```
// Includes JUCE classes
```

```
#include <JuceHeader.h>
```

```
// Includes the JUCE DSP module
```

```
#include <juce_dsp/juce_dsp.h>
```

```
// This is an audio engine class
```

```
class DJAudioPlayer : public juce::AudioSource
```

```
{
```

```
    //public:
```

```
public:
```

```
    // Constructor
```

```
    DJAudioPlayer(juce::AudioFormatManager& formatManager);
```

```
    // Destructor
```

```
    ~DJAudioPlayer();
```

```
    // Called before audio playback begins
```

```
    void prepareToPlay(int samplesPerBlockExpected, double sampleRate)
```

```
override;
```

```
    // Called repeatedly by JUCE to fill the output audio buffer
```

```
    void getNextAudioBlock(const juce::AudioSourceChannelInfo&
```

```
bufferToFill) override;
```

```
    // Free the resources
```

```
    void releaseResources() override;
```

```
    // loads an audio file
```

```
    void loadURL(juce::URL audioURL);
```

```
    // Sets output volume
```

```
    void setGain(double gain);
```

```

// Sets playback speed
void setSpeed(double ratio);

// Sets playback position
void setPositionRelative(double pos);

// Starts playback
void play();

// Stops playback
void stop();

// Gets the relative position
double getPositionRelative();

// Gets the current playback position
double getCurrentPosition() const;

// Sets playback position
void setPosition(double seconds);

*/
/* NOT DONE BY ME
// Sets a hot cue
void setHotCue(int index);

// Jumps to the hot cue
void triggerHotCue(int index);

// Clears all stored hot cues
void clearHotCues();

// Returns all cue positions as an array of 8 values
std::array<double, 8> getHotCues() const;

// Sets cue positions '
void setHotCues(const std::array<double, 8>& cues);

```

```

// Applies Equaliser
void setEQ(double lowGain, double midGain, double highGain);

// Estimate BPM
double estimateBPM(juce::URL audioURL);

*/
/* DONE BY ME
private:

    // 8 cue points all defaulted to -1
    std::array<double, 8> hotCues{ -1, -1, -1, -1, -1, -1, -1, -1 };

    // Used to create AudioFormatReaders
    juce::AudioFormatManager& formatManager;

    // unique_ptr means DJAudioPlayer owns it
    std::unique_ptr<juce::AudioFormatReaderSource> readerSource;

    // Controls transport-style playback
    juce::AudioTransportSource transportSource;

    // ResamplingAudioSource changes playback speed
    juce::ResamplingAudioSource resampleSource{ &transportSource, false, 2
};

*/

/* NOT DONE BY ME
    // A filter that processes float samples
    using IIRFilter = juce::dsp::IIR::Filter<float>;

    // Coefficients define filter shape
    using IIRCoeffs = juce::dsp::IIR::Coefficients<float>;

```

```
// ProcessorDuplicator makes a mono processor work on multi-channel data by duplicating
```

```
using DuplicatedIIR = juce::dsp::ProcessorDuplicator<IIRFilter, IIRCoeffs>;
```

```
// ProcessorChain allows stacking multiple DSP processors
juce::dsp::ProcessorChain<
    DuplicatedIIR,
    DuplicatedIIR,
    DuplicatedIIR
> eqChain;
};
```

```
*/
```

DJAudioPlayer.cpp:

```
/* DONE BY ME
```

```
// Include the DJAudioPlayer class definition
#include "DJAudioPlayer.h"
```

```
// Constructor
```

```
DJAudioPlayer::DJAudioPlayer(juce::AudioFormatManager& _formatManager) :
formatManager(_formatManager)
{
```

```
}
```

```
// Destructor
```

```
DJAudioPlayer::~DJAudioPlayer()
{

}
```

```
// Called before playback starts
```

```

void DJAudioPlayer::prepareToPlay(int samplesPerBlockExpected, double
sampleRate)
{
    // Prepare playback transport
    transportSource.prepareToPlay(samplesPerBlockExpected, sampleRate);

    // Prepare resampling source
    resampleSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
    */
    /* NOT DONE BY ME
    // Setup DSP specification
    juce::dsp::ProcessSpec spec;
    spec.sampleRate = sampleRate;
    spec.maximumBlockSize =
static_cast<juce::uint32>(samplesPerBlockExpected);
    spec.numChannels = 2u;

    // Prepare EQ Filter chain
    eqChain.prepare(spec);

    // Intitalize EQ to neutral
    setEQ(1.0, 1.0, 1.0);
}

// Sets the equalisers
void DJAudioPlayer::setEQ(double lowGain, double midGain, double highGain)
{
    // Sample rate is 44100.0
    double sampleRate = 44100.0;

    // Checks if the reader source is null
    if (readerSource != nullptr)
    {
        sampleRate = readerSource->getAudioFormatReader()->sampleRate;
    }

    // Ensure gain is at least a tiny non-zero value

```

```

float safeLow = std::max(0.001f, (float)lowGain);
float safeMid = std::max(0.001f, (float)midGain);
float safeHigh = std::max(0.001f, (float)highGain);

auto lowCoeffs =
juce::dsp::IIR::Coefficients<float>::makeLowShelf(sampleRate, 200.0f, 0.7f,
safeLow);
auto midCoeffs =
juce::dsp::IIR::Coefficients<float>::makePeakFilter(sampleRate, 1000.0f, 1.0f,
safeMid);
auto highCoeffs =
juce::dsp::IIR::Coefficients<float>::makeHighShelf(sampleRate, 5000.0f, 0.7f,
safeHigh);

*eqChain.get<0>().state = *lowCoeffs;
*eqChain.get<1>().state = *midCoeffs;
*eqChain.get<2>().state = *highCoeffs;
}
*/

/* DONE BY ME
// Repeatedly called to fill audio buffer
void DJAudioPlayer::getNextAudioBlock(const juce::AudioSourceChannelInfo&
bufferToFill)
{
    // Pull audio from transport
    resampleSource.getNextAudioBlock(bufferToFill);

    // Apply EQ only when audio is loaded
    if (readerSource != nullptr)
    {
        // Wrap buffer
        juce::dsp::AudioBlock<float> block(*bufferToFill.buffer);

        auto subBlock = block.getSubBlock((size_t)bufferToFill.startSample,
(size_t)bufferToFill.numSamples);

```

```

        // In-place processing
        juce::dsp::ProcessContextReplacing<float> context(block);

        // Apply equaliser filters
        eqChain.process(context);
    }
}

// Called when audio stops
void DJAudioPlayer::releaseResources()
{
    // Transport buffers released
    transportSource.releaseResources();

    // Resampler buffers released
    resampleSource.releaseResources();
}

// Load audio file
void DJAudioPlayer::loadURL(juce::URL audioURL)
{
    // Create a reader for the file
    auto* reader =
formatManager.createReaderFor(audioURL.createInputStream(false));

    if (reader != nullptr)
    {
        // Wrap reader in AudioFormatReaderSource
        std::unique_ptr<juce::AudioFormatReaderSource> newSource(
            new juce::AudioFormatReaderSource(reader, true));

        // Attach source
        transportSource.setSource(newSource.get(), 0, nullptr,
reader->sampleRate);

        // Reset speed to normal

```



```

        resampleSource.setResamplingRatio(1.0);

        // Transfer ownership
        readerSource.reset(newSource.release());
    }
}

// Sets the volume
void DJAudioPlayer::setGain(double gain)
{
    if (gain < 0 || gain > 1.0)
    {
        std::cout << "Gain should be between 0 and 1" << std::endl;
    }
    else
    {
        // Apply the volume
        transportSource.setGain(gain);
    }
}

// Set the playback speed
void DJAudioPlayer::setSpeed(double ratio)
{
    if (ratio < 0.5 || ratio > 2.0)
    {
        return;
    }

    // Apply the speed change
    resampleSource.setResamplingRatio(ratio);
}

// Return the current position
double DJAudioPlayer::getCurrentPosition() const
{
    return transportSource.getCurrentPosition();
}

```

```
}
```

```
// Jump to specific time
```

```
void DJAudioPlayer::setPosition(double seconds)
```

```
{
```

```
    transportSource.setPosition(seconds);
```

```
}
```

```
*/
```

```
/* DONE BY ME
```

```
// Store hot cue
```

```
void DJAudioPlayer::setHotCue(int index)
```

```
{
```

```
    if (index < 0 || index >= hotCues.size())
```

```
    {
```

```
        return;
```

```
    }
```

```
    hotCues[index] = getCurrentPosition();
```

```
}
```

```
// Trigger hot cue
```

```
void DJAudioPlayer::triggerHotCue(int index)
```

```
{
```

```
    if (index < 0 || index >= hotCues.size())
```

```
    {
```

```
        return;
```

```
    }
```

```
    double cuePosition = hotCues[index];
```

```
    if (cuePosition >= 0.0)
```

```
    {
```

```
        // jump to the cue
```

```
        setPosition(cuePosition);
```

```

    }
    else
    {
        // Create cue
        setHotCue(index);
    }
}

// Clear all hot cues
void DJAudioPlayer::clearHotCues()
{
    for (auto& c : hotCues)
    {
        c = -1.0;
    }
}

// Return all cues
std::array<double, 8> DJAudioPlayer::getHotCues() const
{
    return hotCues;
}

// Restore cues from saved data
void DJAudioPlayer::setHotCues(const std::array<double, 8>& cues)
{
    hotCues = cues;
}
*/

/* DONE BY ME
// Set playback position
void DJAudioPlayer::setPositionRelative(double pos)
{
    if (pos < 0 || pos > 1.0)
    {
        std::cout << "Position must be between 0 and 1" << std::endl;
    }
}

```

```

    }
    else
    {
        double posInSecs = transportSource.getLengthInSeconds() * pos;
        setPosition(posInSecs);
    }
}

```

```

// Start the playback
void DJAudioPlayer::play()
{
    transportSource.start();
}

```

```

// Stop the playback
void DJAudioPlayer::stop()
{
    transportSource.stop();
}

```

```

// Return relative playhead position
double DJAudioPlayer::getPositionRelative()
{
    double len = transportSource.getLengthInSeconds();
    if (len <= 0.0)
    {
        return 0.0;
    }

    return transportSource.getCurrentPosition() / len;
}

```

```

*/

```

```

/* NOT DONE BY ME

```

```

// A BPM Estimation using peak detection
double DJAudioPlayer::estimateBPM(juce::URL audioURL)

```

```

{
    auto* reader =
formatManager.createReaderFor(audioURL.createInputStream(false));

    if (!reader)
    {
        // fallback
        return 120.0;
    }

    juce::AudioBuffer<float> buffer(reader->numChannels,
reader->lengthInSamples);
    reader->read(&buffer, 0, reader->lengthInSamples, 0, true, true);

    double sampleRate = reader->sampleRate;

    const float* channelData = buffer.getReadPointer(0);

    float threshold = 0.7f;
    int minSamplesBetweenPeaks = (int)(sampleRate * 0.2);
    int lastPeak = -minSamplesBetweenPeaks;

    std::vector<double> intervals;

    for (int i = 0; i < buffer.getNumSamples(); ++i)
    {
        if (std::abs(channelData[i] > threshold && (i - lastPeak) >
minSamplesBetweenPeaks))
        {
            if (lastPeak >= 0)
            {
                intervals.push_back((i - lastPeak) / sampleRate);
            }

            lastPeak = i;
        }
    }
}

```

```

    if (intervals.empty())
    {
        return 120.0;
    }

    double avg = 0.0;

    for (double v : intervals)
    {
        avg += v;
    }

    avg /= intervals.size();

    double bpm = 60.0 / avg;

    if (bpm < 60 || bpm > 200)
    {
        return 120.0;
    }

    return bpm;
}
*/

```

WaveformDisplay.h:

```

/* DONE BY ME BECAUSE IT WAS IN THE COURSE CONTENT
// This helps ensure the header file isn't included more than once
#pragma once

// This is a JUCE header file that contains the classes and functions in JUCE
library
#include <JuceHeader.h>

```

```
//=====
=====
```

```
// A class that inherits from the Component and Change Listener Class in JUCE
class WaveformDisplay : public juce::Component, public juce::ChangeListener
{
// The public interface of the class
public:
```

```
    // This is a constructor that declares the dependencies of a class which are the
    audio format manager and the audio thumbnail
```

```
    WaveformDisplay(juce::AudioFormatManager& formatManagerNeeded,
juce::AudioThumbnailCache& cacheNeeded);
```

```
    // Overrides the default destructor
    ~WaveformDisplay() override;
```

```
    // The method used to draw the waveform
    void paint(juce::Graphics&) override;
```

```
    // Resizes the component
    void resized() override;
```

```
    // Load a file from a URL
    void loadURL(juce::URL audioURL);
```

```
    // The audio callback triggers when the thumbnail changes
    void changeListenerCallback(juce::ChangeBroadcaster* source) override;
```

```
    // Set the playhead position as a 0-1 value
    void setPositionRelative(double pos);
```

```
// Begin the private data members
private:
```

```
    // The playhead position which is a value between 0 and 1.
    double position;
```

```
// This is the audio thumbnail object
juce::AudioThumbnail audioThumbnail;
```

```
// Flag to check whether a file has loaded or not.
bool fileLoaded;
```

// Prevents copying and checks for leaks within the class. This is a MACRO function.

```
JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(WaveformDisplay
)
};
*/
```

WaveformDisplay.cpp:

```
/* DONE BY ME
```

```
// include the JUCE framework
#include <JuceHeader.h>
```

```
// include the WaveformDisplay header file.
#include "WaveformDisplay.h"
```

```
// includes math utilities
#include <cmath>
```

```
// Inheritance and initialization of states as well as construction
WaveformDisplay::WaveformDisplay(juce::AudioFormatManager&
formatManagerNeeded, juce::AudioThumbnailCache& cacheNeeded) :
audioThumbnail(1000, formatManagerNeeded, cacheNeeded),
fileLoaded(false), position(0)
{
    // registration of this component to receive thumbnail change events
    audioThumbnail.addChangeListener(this);
}
```



```

// The WaveFormDisplayDestructor
WaveformDisplay::~WaveformDisplay()
{

}

// Draw the waveform display
void WaveformDisplay::paint(juce::Graphics& design)
{
    // Fill the background with the window color

design.fillAll(getLookAndFeel().findColour(juce::ResizableWindow::backgroundC
olourId));

    // Set the drawing colour to grey
design.setColour(juce::Colours::grey);

    // Draw the border around the component
design.drawRect(getLocalBounds(), 1);

    // Set the drawing colour to orange
design.setColour(juce::Colours::orange);

    // Checks if an audio file has loaded
    if (fileLoaded)
    {
        // Draw the waveforms for the audio that's been loaded
        audioThumbnail.drawChannel(design, getLocalBounds(), 0,
audioThumbnail.getTotalLength(), 0, 1.0f);

        // Sets the colour for the playhead
design.setColour(juce::Colours::lightgreen);

        // Converts the relative position to pixel X coordinate
        const int pixel = juce::jlimit(0, getWidth(), juce::roundToInt(position *
(double)getWidth()));

```

```

// Calculation of the playhead width with a minimum of 1 pixel
const int rectW = std::max(1, getWidth() / 20);

// Draws playback rectangle over the waveform
design.drawRect(pixel, 0, rectW, getHeight());
}
else // Happens when no file is loaded
{
    // Sets the font size of the placeholder text
    design.setFont(20.0f);

    // Center placeholder message is displayed
    design.drawText("File not loaded...", getLocalBounds(),
juce::Justification::centred, true);
}
}

// This function is called when the component size changes
void WaveformDisplay::resized()
{

}

// This loads audio data from the URL
void WaveformDisplay::loadURL(juce::URL audioURL)
{
    // Any previously loaded waveform data is cleared
    audioThumbnail.clear();

    // A new audio source is loaded and the loaded state is updated
    fileLoaded = audioThumbnail.setSource(new
juce::URLInputSource(audioURL));
}

void WaveformDisplay::changeListenerCallback(juce::ChangeBroadcaster*
source)

```

```

{
    // It triggers the repaint to update the waveform
    repaint();
}

// The playhead position is updated using the relative value
void WaveformDisplay::setPositionRelative(double pos)
{
    // Checks if there is not a number or an infinity value
    if (!std::isfinite(pos))
    {
        // An invalid position is set to 0
        pos = 0.0;
    }

    // The position is clamped between 0 and 1
    pos = juce::jlimit(0.0, 1.0, pos);

    // Checks if the playhead position has changed
    if (pos != position)
    {
        // Stores a new playhead position
        position = pos;

        // A waveform is redrawn with an updated playhead
        repaint();
    }
}
*/

```

DeckGUI.h:

```

/* DONE BY ME
// Prevents this header from being included more than once
#pragma once

// Include the JUCE Header

```

```

#include <JuceHeader.h>

// Audio playback engine
#include "DJAudioPlayer.h"

// Draws the Waveform
#include "WaveformDisplay.h"

//=====

=====

// Represents one deck
class DeckGUI : public juce::Component, public juce::Button::Listener, public
juce::Slider::Listener, public juce::FileDragAndDropTarget, public juce::Timer
{
public:

    // Constructor
    DeckGUI(DJAudioPlayer* player,
            juce::AudioFormatManager& formatManagerToUse,
            juce::AudioThumbnailCache& cacheToUse);

    // Destructor
    ~DeckGUI();

    // Paint method
    void paint(juce::Graphics&) override;

    // Called when the component is resized
    void resized() override;

    // Called when any button in the component is clicked
    void buttonClicked(juce::Button*) override;

    // Called when a slider value is changed
    void sliderValueChanged(juce::Slider* slider) override;

```

```

// Checks if dragged files are accepted
bool isInterestedInFileDrag(const juce::StringArray& files) override;

// Called when files are dropped into the component
void filesDropped(const juce::StringArray& files, int x, int y) override;

// Timer callback
void timerCallback() override;

// Loads the file into the deck
void loadFile(juce::URL audioURL);
*/
/* NOT DONE BY ME
// Sets deck index so callbacks know which deck it is
void setDeckIndex(int index);

// Set callbacks for cue handling
void setCueCallbacks(
    std::function<void(const juce::String&, int, const std::array<double,
8>&)> cuesChanged,
    std::function<void(const juce::String&, int, DJAudioPlayer*)>
trackLoaded
);

// Callback triggered when equaliser sliders change
std::function<void(const juce::String&, double, double, double)>
onEQChanged;

private:

// Play track
juce::TextButton playButton{ "PLAY" };

// Stop track
juce::TextButton stopButton{ "STOP" };

// Open file chooser

```

```

juce::TextButton loadButton{ "LOAD" };

// Clear all hot cues
juce::TextButton clearCuesButton{ "Clear Cues" };

// Array of 8 cues
std::array<juce::TextButton, 8> hotCueButtons
{
    juce::TextButton{ "C1" },
    juce::TextButton{ "C2" },
    juce::TextButton{ "C3" },
    juce::TextButton{ "C4" },
    juce::TextButton{ "C5" },
    juce::TextButton{ "C6" },
    juce::TextButton{ "C7" },
    juce::TextButton{ "C8" },
};

// Path of the current track thats loaded
juce::String loadedTrackPath;

// Callback triggered when cues changed
std::function<void(const juce::String& trackPath, int deckIndex, const
std::array<double, 8>& cues)> onCuesChanged;

// Callback triggered when the track is loaded
std::function<void(const juce::String& trackPath, int deckIndex,
DJAudioPlayer* player)> onTrackLoaded;

// Callback triggered when a track is loaded
int deckIndex = 1;

// Volume slider
juce::Slider volSlider;

// Playback speed
juce::Slider speedSlider;

```

```

        // 3 Band EQ sliders
        juce::Slider lowSlider;
        juce::Slider midSlider;
        juce::Slider highSlider;

        // Displays detected BPM value
        juce::Label bpmLabel;

        // Current BPM
        double currentBPM = 120.0;
    */
    /* DONE BY ME
        // Track position
        juce::Slider posSlider;

        // File chooser
        juce::FileChooser fChooser{ "Select a file..." };

        // Points to the audio engine
        DJAudioPlayer* player;

        // Visual waveform display
        WaveformDisplay waveformDisplay;

        JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(DeckGUI)
    };
    */

```

DeckGUI.cpp:

```

    /* DONE BY ME
    // include the JuceHeader file
    #include <JuceHeader.h>

    // Include the DeckGUI header file

```

```
#include "DeckGUI.h"
```

```
//=====
=====
```

```
// Constructor
```

```
DeckGUI::DeckGUI(DJAudioPlayer* _player, juce::AudioFormatManager&
formatManagerToUse, juce::AudioThumbnailCache& cacheToUse) :
player(_player), waveformDisplay(formatManagerToUse, cacheToUse)
{
```

```
    // Add UI elements to the component
```

```
    addAndMakeVisible(playButton);
    addAndMakeVisible(stopButton);
    addAndMakeVisible(loadButton);
    addAndMakeVisible(volSlider);
    addAndMakeVisible(speedSlider);
    addAndMakeVisible(posSlider);
    addAndMakeVisible(waveformDisplay);
    addAndMakeVisible(clearCuesButton);
```

```
    // Buttons and Volume sliders
```

```
    playButton.addListener(this);
    stopButton.addListener(this);
    loadButton.addListener(this);
    volSlider.addListener(this);
    speedSlider.addListener(this);
    posSlider.addListener(this);
```

```
    /*
```

```
    /* NOT DONE BY ME
```

```
    clearCuesButton.addListener(this);
```

```
    // Add and make visible the equaliser sliders and the BPM label
```

```
    addAndMakeVisible(lowSlider);
    addAndMakeVisible(midSlider);
    addAndMakeVisible(highSlider);
    addAndMakeVisible(bpmLabel);
```

```
    // Add listeners to the sliders
```



```

lowSlider.addListener(this);
midSlider.addListener(this);
highSlider.addListener(this);

// Set initial text
bpmLabel.setText("BPM: --", juce::dontSendNotification);

// Make the Hot Cue buttons visible
for (auto& btn : hotCueButtons)
{
    addAndMakeVisible(btn);
    btn.addListener(this);
}

// Volume 0 to 1
volSlider.setRange(0.0, 1.0);

// Speed Range as well as default speed
speedSlider.setRange(0.5, 2.0, 0.01);
speedSlider.setValue(1.0);

// Relative position 0..1
posSlider.setRange(0.0, 1.0);

// Equaliser sliders
lowSlider.setRange(0.0, 2.0, 0.01);
midSlider.setRange(0.0, 2.0, 0.01);
highSlider.setRange(0.0, 2.0, 0.01);

// Default Equaliser values = 1.0
lowSlider.setValue(1.0);
midSlider.setValue(1.0);
highSlider.setValue(1.0);

// Start a timer that calls timerCallback() every 200ms
startTimer(200);
}

```

```

*/
/* DONE BY ME
// Destructor
DeckGUI::~DeckGUI()
{
    // Stop timer
    stopTimer();
}

// Paint draws the background
void DeckGUI::paint(juce::Graphics& design)
{
    // Fill the component background

    design.fillAll(getLookAndFeel().findColour(juce::ResizableWindow::backgroundC
olourId));

    design.drawRect(getLocalBounds(), 1);

    // Set drawing colour and font
    design.setColour(juce::Colours::white);
    design.setFont(14.0f);

    // Draw label text in the centre
    design.drawText("DeckGUI", getLocalBounds(), juce::Justification::centred,
true);
}

// Resize the UI layout
void DeckGUI::resized()
{
    // Compute a row height and 4 equal columns
    double rowH = getHeight() / 13.0;
    double colW = getWidth() / 4.0;

    // First row
    playButton.setBounds(0, 0, getWidth() / 3, rowH);

```

```

stopButton.setBounds(getWidth() / 3, 0, getWidth() / 3, rowH);
loadButton.setBounds(2 * getWidth() / 3, 0, getWidth() / 3, rowH);

// Next row
volSlider.setBounds(0, rowH, getWidth(), rowH);
speedSlider.setBounds(0, rowH * 2, getWidth(), rowH);
posSlider.setBounds(0, rowH * 3, getWidth(), rowH);

// Wave form holds 2 rows
waveformDisplay.setBounds(0, rowH * 4, getWidth(), rowH * 2);
*/
/* NOT DONE BY ME
// Hot cue buttons
for (int i = 0; i < 4; i++)
{
    hotCueButtons[i].setBounds(i * colW, rowH * 6, colW, rowH);
    hotCueButtons[i + 4].setBounds(i * colW, rowH * 7, colW, rowH);
}

// EQ sliders
double eqW = getWidth() / 3.0;
lowSlider.setBounds(0, rowH * 8, eqW, rowH);
midSlider.setBounds(eqW, rowH * 8, eqW, rowH);
highSlider.setBounds(2 * eqW, rowH * 8, eqW, rowH);

// Button to clear cues
clearCuesButton.setBounds(0, rowH * 9, getWidth(), rowH);

// BPM label
bpmLabel.setBounds(0, rowH * 10, getWidth(), rowH);
}
*/
/* DONE BY ME
void DeckGUI::buttonClicked(juce::Button* button)
{
    // If play was clicked
    if (button == &playButton)

```

```

{
    player->play();
}
// If stop was clicked, stop playback
else if (button == &stopButton)
{
    player->stop();
}
// if Load was clicked, open file chooser
else if (button == &loadButton)
{
    // Allow selecting files only
    auto fileChooserFlags = juce::FileBrowserComponent::canSelectFiles;

    // Launch async chooser so UI doesn't freeze
    fChooser.launchAsync(fileChooserFlags, [this](const juce::FileChooser&
chooser)
    {
        // Get the selected file
        auto file = chooser.getResult();

        // if user picked a real file, load it
        if (file.existsAsFile())
            loadFile(juce::URL{ file });
    });
}
*/
/* NOT DONE BY ME
// Check if a hot cue button was clicked
for (int i = 0; i < (int)hotCueButtons.size(); ++i)
{
    if (button == &hotCueButtons[(size_t)i])
    {
        // trigger cue i on the player
        player->triggerHotCue(i);

        if (onCuesChanged && loadedTrackPath.isNotEmpty())

```

```

    {
        // Save current cues to JSON
        onCuesChanged(loaderTrackPath, deckIndex, player->getHotCues());
    }

    // Return
    return;
}

// If clear cue was clicked
if (button == &clearCuesButton)
{
    // remove all stored cues in the player
    player->clearHotCues();

    // Notify parent so it can save cleared cues
    if (onCuesChanged && loaderTrackPath.isNotEmpty())
    {
        onCuesChanged(loaderTrackPath, deckIndex, player->getHotCues());
    }
}

// Called when a slider with this listener changes
void DeckGUI::sliderValueChanged(juce::Slider* slider)
{
    // If an EQ slider moved, apply EQ
    if (slider == &lowSlider || slider == &midSlider || &highSlider)
    {
        // Apply EQ settings
        player->setEQ(lowSlider.getValue(), midSlider.getValue(),
highSlider.getValue());

        // Checks if the EQ changed and the loaded track isnt empty
        if (onEQChanged && loaderTrackPath.isNotEmpty())
        {

```

```

        onEQChanged(loadedTrackPath, lowSlider.getValue(),
midSlider.getValue(), highSlider.getValue());
    }
*/
/* DONE BY ME
    // Volume slider controls the gain
    if (slider == &volSlider)
    {
        player->setGain(slider->getValue());
    }

    // Speed Slider controls the playback speed
    if (slider == &speedSlider)
    {
        player->setSpeed(slider->getValue());
    }

    // Position slider controls
    if (slider == &posSlider)
    {
        player->setPositionRelative(slider->getValue());
    }
}
}

// Asks if the dragged files are acceptable
bool DeckGUI::isInterestedInFileDrag(const juce::StringArray& files)
{
    return true;
}

void DeckGUI::loadFile(juce::URL audioURL)
{
    // Load audio into the player
    player->loadURL(audioURL);

    // Load audio into the waveform display

```

```

    waveformDisplay.loadURL(audioURL);

    /*
    /* NOT DONE BY ME
    // Estimate the BPM
    double estimatedBpm = player->estimateBPM(audioURL);

    // Update the BPM Label
    bpmLabel.setText("BPM: " + juce::String(estimatedBpm, 1),
        juce::dontSendNotification);

    // If the URL points to the local file, store its path
    if (audioURL.isLocalFile())
    {
        loadedTrackPath = audioURL.getLocalFile().getFullPathName();

        // Notify the parent that a track was loaded
        if (onTrackLoaded)
        {
            onTrackLoaded(loadedTrackPath, deckIndex, player);
        }
    }
}
/*
/* DONE BY ME
// Calls this when files are dropped
void DeckGUI::filesDropped(const juce::StringArray& files, int x, int y)
{
    if (files.size() == 1)
    {
        loadFile(juce::URL{ juce::File{ files[0] } });
    }
}
// Called periodically
void DeckGUI::timerCallback()
{
    waveformDisplay.setPositionRelative(player->getPositionRelative());
}

```

```

}

// Stores the deck number
void DeckGUI::setDeckIndex(int index)
{
    deckIndex = index;
}

*/
/* NOT DONE BY ME
// Pass callbacks into DeckGUI
void DeckGUI::setCueCallbacks(
    std::function<void(const juce::String&, int, const std::array<double, 8>&)>
    cuesChanged,
    std::function<void(const juce::String&, int, DJAudioPlayer*)> trackLoaded)
{
    // Save the callbacks for later use
    onCuesChanged = std::move(cuesChanged);
    onTrackLoaded = std::move(trackLoaded);
}
*/

```

PlaylistComponent.h:

```

/* DONE BY ME SINCE IT WAS IN THE COURSE CONTENT
// This helps ensure the header file isn't included more than once
#pragma once

// The main JUCE header that contains all the JUCE classes and functions
#include <JuceHeader.h>

// Provides the std::vector container
#include <vector>

// Includes the std::string type
#include <string>

```



```
//=====
=====
```

```
// A UI component that acts like a table model and a listener
```

```
class PlaylistComponent : public juce::Component, public
juce::TableListBoxModel, public juce::Button::Listener
{
public:
```

```
    // A constructor that takes the audio format manager as a reference
    PlaylistComponent(juce::AudioFormatManager& formatManager);
```

```
    // Declaration of the destructor
    ~PlaylistComponent() override;
```

```
    // The method that draws the component is declared
    void paint(juce::Graphics&) override;
```

```
    // The method that resizes the component is declared
    void resized() override;
```

```
    // The number of rows is returned by this method
    int getNumRows();
```

```
    // The background of a table row is painted
    void paintRowBackground(juce::Graphics&, int rowNumber, int width, int
height, bool rowIsSelected) override;
```

```
    // An individual table cell is painted
    void paintCell(juce::Graphics&, int rowNumber, int columnId, int width, int
height, bool rowIsSelected) override;
```

```
    // Method which handles a click on the table cell
    Component* refreshComponentForCell(int rowNumber, int columnId, bool
isRowSelected, Component* existingComponentToUpdate) override;
```

```
    // When the listened button is clicked, this method is executed
```

```

void buttonClicked(juce::Button* button) override;

// A callback is stored to load a selected file into a certain deck
std::function<void(juce::File, int)> onLoadToDeck;

// Metadata for one track
struct Track {
    juce::File file;
    juce::String fileName;
    juce::String duration;
    double lengthInSeconds;
};

// A helper that converts seconds into a mm:ss style string
static juce::String secondToMinutes(double seconds);

// Get the library file
juce::File getLibraryFile() const;

// Loads the playlist library
void loadLibrary();

// Saves the playlist library without object modification
void saveLibrary() const;

private:

// Import a track button
juce::TextButton importButton{ "Import Track" };

// A UI component that displays track rows
juce::TableListBox tableComponent;

// Stores the list of tracks
std::vector<Track> tracks;

// Stores a reference to the audio format manager

```

```

juce::AudioFormatManager& formatManager;

// A file chooser dialog
juce::FileChooser fChooser{ "Select files..." };

// Disables copying and enables JUCE leak detection

JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(PlaylistComponent
t)
};
*/

```

PlaylistComponent.cpp:

```

/* DONE BY ME
// include the JUCE framework
#include <JuceHeader.h>

// Includes the PlaylistComponent header file
#include "PlaylistComponent.h"

//=====
=====

// Construction of the component and storage of the reference to the audio
format manager
PlaylistComponent::PlaylistComponent(juce::AudioFormatManager&
_formatManager) : formatManager(_formatManager)
{
    // Adds the import button and makes it visible
    addAndMakeVisible(importButton);

    // This class is registered to receive click events from the import button
    importButton.addListener(this);

    // Adds the table component to the UI
    addAndMakeVisible(tableComponent);
}
*/

```

/* NOT DONE BY ME

```
    // Header flags for resizable and visible columns
    auto titleFlags = juce::TableHeaderComponent::visible |
juce::TableHeaderComponent::resizable;

    // Header flags for button columns
    auto buttonFlags = juce::TableHeaderComponent::visible;

    // The table components
    tableComponent.getHeader().addColumn("Track Title", 1, 200, 100, -1,
titleFlags);
    tableComponent.getHeader().addColumn("Duration", 2, 80, 50, 100,
buttonFlags);
    tableComponent.getHeader().addColumn("Deck 1", 3, 70, 50, 100,
buttonFlags);
    tableComponent.getHeader().addColumn("Deck 2", 4, 70, 50, 100,
buttonFlags);

    tableComponent.getHeader().setStretchToFitActive(true);
    tableComponent.setModel(this);
```

}

***/**

/* DONE BY ME

```
// The Destructor that is used for clean up
PlaylistComponent::~PlaylistComponent()
{

}
```

```
// Paints the component's background
void PlaylistComponent::paint(juce::Graphics& design)
{
```

```
    // Fills the background with the window colour
```

```
design.fillAll(getLookAndFeel().findColour(juce::ResizableWindow::backgroundC
olourId));
```

```

// The drawing colour is set to grey
design.setColour(juce::Colours::grey);

// Sets the font size
design.setFont(juce::FontOptions(14.0f));

// The placeholder componet name text
design.drawText("PlaylistComponent", getLocalBounds(),
juce::Justification::centred, true);
}

void PlaylistComponent::resized()
{
    // Positions the import button at the top
    importButton.setBounds(0, 0, getWidth(), 30);

    // Positions the table below the button
    tableComponent.setBounds(0, 30, getWidth(), getHeight() - 30);
}

int PlaylistComponent::getNumRows()
{
    // Returns the amount of tracks that are in the playlist
    return tracks.size();
}

// Paint the text content of an individual cell
void PlaylistComponent::paintCell(juce::Graphics& design, int rowNumber, int
columnId, int width, int height, bool rowIsSelected)
{
    // Checks if the row index is valid
    if (rowNumber < tracks.size())
    {
        // Checks if the current column is the Track Title column
        if (columnId == 1)
        {
            // Draws the track file name in the cell

```

```

        design.drawText(tracks[rowNumber].fileName, 2, 0, width - 4,
height, juce::Justification::centredLeft, true);
    }
    // Checks if the current column is the DURATION column
    else if (columnId == 2)
    {
        // Draws the track's duration text in the cell
        design.drawText(tracks[rowNumber].duration, 2, 0, width - 4,
height, juce::Justification::centredLeft, true);
    }
}
}

```

```

// Paints the background behind each row
void PlaylistComponent::paintRowBackground(juce::Graphics& design, int
rowNumber, int width, int height, bool rowIsSelected)
{
    // Checks if the row is selected
    if (rowIsSelected)
    {
        // Fills the selected row background with orange
        design.fillAll(juce::Colours::orange);
    }
    // Handles the non-selected rows
    else
    {
        // Fills the non-selected row background with dark grey
        design.fillAll(juce::Colours::darkgrey);
    }
}
}

```

```

// Creates or updates the custom component inside a table cell
juce::Component* PlaylistComponent::refreshComponentForCell(int rowNumber,
int columnId, bool isRowSelected, Component* existingComponentToUpdate)
{
    // Checks if the cell is in one of the deck button columns
    if (columnId == 3 || columnId == 4)

```

```

{
    // Reuse the existing component as a text button
    juce::TextButton* btn =
dynamic_cast<juce::TextButton*>(existingComponentToUpdate);

    // if no reusable button exists
    if (btn == nullptr)
    {
        // Allocates a new button labeled "Load"
        btn = new juce::TextButton("Load");

        // Registers this class to receive click events
        btn->addListener(this);

        // Updates the returned component pointer
        existingComponentToUpdate = btn;
    }

    // A unique ID string encoding row and column
    juce::String id{ std::to_string(rowNumber) + "_" +
std::to_string(columnId) };

    // Stores the ID on the button
    btn->setComponentID(id);

    // Sets the button bounds with padding
    btn->setBounds(2, 2,
tableComponent.getHeader().getColumnWidth(columnId) - 4,
tableComponent.getRowHeight() - 4);
    }

    // Returns the component JUCE should place in this cell
    return existingComponentToUpdate;
}

// Handles click events
void PlaylistComponent::buttonClicked(juce::Button* button)

```

```

{
    // Checks if the clicked button is the import button
    if (button == &importButton)
    {
        // Sets file chooser to allow selecting files
        auto fileChooserFlags = juce::FileBrowserComponent::canSelectFiles
| juce::FileBrowserComponent::canSelectMultipleItems;

        // Opens the file chooser asynchronously and runs a callback
        fChooser.launchAsync(fileChooserFlags, [this](const
juce::FileChooser& chooser)
        {
            // Gets the list of files the user had selected
            auto results = chooser.getResults();

            // Loops through each selected file
            for (auto& file : results)
            {
                // Creates an audio reader
                std::unique_ptr<juce::AudioFormatReader>
reader(formatManager.createReaderFor(file));

                // Checks if the reader was succesfully created
                if (reader != nullptr)
                {

                    double length = reader->lengthInSamples /
reader->sampleRate;

                    // Creates a Track struct with file
                    properties
                    Track newTrack{ file, file.getFileName(),
secondToMinutes(length), length };

                    // Adds the new track to the playlist vector
                    tracks.push_back(newTrack);
                }
            }
        }
    }
}

```



```

    }

    // Refreshes the table
    tableComponent.updateContent();

    // Saves the updated playlist
    saveLibrary();
});
}
// Handles clicks from dynamically created "Load" buttons
else
{
    // Reads the button's ID string
    juce::String id = button->getComponentID();

    // Finds the underscore separating row and column
    int underScorePos = id.indexOf("_");

    // Checks that the underscore was found
    if (underScorePos > -1)
    {
        // Parses the row index
        int row = id.substring(0, underScorePos).getIntValue();

        // Parses the column ID after the understore to the end
        int col = id.substring(underScorePos + 1).getIntValue();

        // Validates that the row index is within the vector bounds
        if (row >= 0 && row < tracks.size())
        {
            // Converts column ID 3 into a deck number which is
            either 1 or 2

            int deckIndex = (col == 3) ? 1 : 2;

            // Checks that a load callback has been assigned
            if (onLoadToDeck)
            {

```

```

// Calls the callback with the chosen file and the
target deck
onLoadToDeck(tracks[row].file, deckIndex);
    }
    }
    }
}
*/
/* NOT DONE BY ME
// Converts a duration into seconds
juce::String PlaylistComponent::secondToMinutes(double seconds)
{
    // Casts seconds to an integer
    int secondsInt = (int)seconds;

    // Computes whole minutes from the total seconds
    int minutes = secondsInt / 60;

    // Computes leftover seconds
    int remainingSeconds = secondsInt % 60;

    // Returns zero-padded mm:ss string
    return juce::String(minutes) + ":" + ((remainingSeconds < 10) ? "0" : "") +
juce::String(remainingSeconds);
}

// Returns the file path where the playlist library JSON is
juce::File PlaylistComponent::getLibraryFile() const
{
    // Builds an app specific deirectory path
    auto appDataDir =
juce::File::getSpecialLocation(juce::File::userApplicationDataDirectory).getChildF
ile(ProjectInfo::projectName);

    // Checks whether the application data directory exists
    if (!appDataDir.exists())

```

```

{
    // Creates the directory if its shown that it doesnt exist
    appDataDir.createDirectory();
}

// Returns an aobject pointing to the library.json file
return appDataDir.getChildFile("library.json");
}

// Saves the current tracks vector into a JSOn file
void PlaylistComponent::saveLibrary() const
{
    // Array of var objects
    juce::Array<juce::var> tracksList;

    // Iterates over each track in the playlist
    for (const auto& track : tracks)
    {
        // Allocates a JSON-like dynamic object
        auto* object = new juce::DynamicObject();

        // Creates objects out of the file properties
        object->setProperty("path", track.file.getFullPathName());
        object->setProperty("fileName", track.fileName);
        object->setProperty("lengthInSeconds", track.lengthInSeconds);
        object->setProperty("duration", track.duration);

        // Adds that object to the tracksList
        tracksList.add(object);
    }

    // Wraps the array into a root var value
    juce::var root(tracksList);

    // Serializes the var structure into a JSON string
    auto libraryJson = juce::JSON::toString(root);

```

```

        // Gets the output JSON file path
        auto libraryFile = getLibraryFile();

        // Writes the JSON string to the file
        libraryFile.replaceWithText(libraryJson);
    }

// Loads the tracks from the library JSON file
void PlaylistComponent::loadLibrary()
{
    // Clears the current tracks
    tracks.clear();

    // Gets the file path
    auto libraryFile = getLibraryFile();

    // Checks if the library file exists
    if (!libraryFile.existsAsFile())
    {
        // Refreshes the table
        tableComponent.updateContent();

        // Exits early
        return;
    }

    // Loads the entire JSON file into string
    auto text = libraryFile.loadFileAsString();

    // Parses the string into a JSON var structure
    auto parsed = juice::JSON::parse(text);

    // Checks that the parsed JSON is an array
    if (!parsed.isArray())
    {
        // Refreshes the table
        tableComponent.updateContent();
    }
}

```

```

        // Exits early
        return;
    }

    // Gets a pointer to the underlying array
    auto* array = parsed.getJSONArray();

    // Iterates through each item in the JSON list
    for (const auto& item : *array)
    {
        // Checks that the object is a JSON object
        if (auto* object = item.getDynamicObject())
        {
            // reads the stored file path from the JSON object
            const auto path = object->getProperty("path").toString();

            // Creates a JUCE file from that path
            juce::File file(path);

            // Skips the entries of files that dont exist on disk
            if (!file.existsAsFile())
            {
                // Continues to the next item without adding the track
                continue;
            }

            // Empty track created that is populated with JSON
            Track track1;

            // File reference stored in that track
            track1.file = file;

            // Reads the saved filename from that JSON
            track1.fileName = object->getProperty("fileName").toString();

            // Reads the saved numeric length from JSON

```

```

        track1.lengthInSeconds =
object->getProperty("lengthInSeconds");

        // Reads the saved formatted duration string from JSON
track1.duration = object->getProperty("duration").toString();

        // Checks if the stored filename is missing
if (track1.fileName.isEmpty())
{
    // Uses the actual file name from disk
    track1.fileName = file.getFileName();
}

        // Checks if the stored duration string is missing
if (track1.duration.isEmpty())
{
    // Checks the duration string
    track1.duration =
secondToMinutes(track1.lengthInSeconds);
}

        // Adds the track to the playlist vector
tracks.push_back(track1);
    }
}

// Updates the table
tableComponent.updateContent();
}
*/

```